

Here are the class diagrams represented in a structured text format. I have defined the entities (classes), their properties (attributes/inputs), and their behaviors (methods/outcomes) based on your specifications. I also included the specific data structures (like DiseaseReport or RecommendationJSON) that these models would pass between each other.

## 1. CropDiseaseDetector

This class handles the multimodal input (image + text) to diagnose plant health and prescribe treatments.

### Attributes (Inputs):

- + cropImage: Image
- + textualDescription: String

### Methods (Operations):

- + detectDisease(): DiseaseReport
  - *Returns* DiseaseReport: An object containing detectedDiseaseName (String), recommendedSolutions (List), medications (List), and requiredFertilizers (List).

## Crop Disease Detection

### Actors Involved:

- Farmer (Initiator)
- Diagnostic AI Engine (Processor)
- Treatment & Medication Database (Data Source)

### Inputs:

- Image of the affected crop
- Textual description of symptoms

### Functions (The Flow):

1. The Farmer captures an image of the plant anomaly and types a brief description.

2. The System pre-processes the image for computer vision and tokenizes the textual description.
3. The Diagnostic AI Engine analyzes the multimodal data to classify the disease and assign a confidence score.
4. The System queries the Treatment Database using the identified disease name.
5. The System compiles the matched chemical, biological, and fertilizer solutions into a cohesive report.

**Outputs:**

Disease Report (Detected disease, recommended solutions, required medications, and specific fertilizers).

## 2. CropRecommender

This class processes environmental, physical, and economic parameters to suggest the most optimal crop.

### Attributes (Inputs):

- + soilType: String
- + soilMoistureProfile: Float
- + weather: WeatherData
- + temperature: Float
- + cropDemand: MarketMetric
- + importExportInfo: TradeData
- + highestMargin: Float
- + effortLevel: String (e.g., Low, Medium, High)

### Methods (Operations):

- + generateRecommendation(): JSON
  - *Returns* JSON: A structured payload containing the recommendedCrop (String) and selectionReasoning (String detailing why it was chosen based on the inputs).

## Crop Recommendation

### Actors Involved:

- Farmer / Farm Manager (Initiator)
- External APIs (Weather and Market Data Sources)
- Recommendation AI (Processor)

### Inputs:

- Soil type and moisture profile
- Local weather and temperature forecasts
- Crop demand and import/export data
- Desired profit margin and effort level capacity

**Functions (The Flow):**

1. The Farmer inputs local environmental constraints and personal business preferences.
2. The System fetches real-time market data (demand, trade) and weather forecasts via External APIs.
3. The Recommendation AI weighs the environmental data against the economic and effort constraints.
4. The System ranks the most viable crops, prioritizing the highest margin score that fits the effort level.
5. The System packages the top recommendation and the logical reasoning into a structured format.

**Outputs:**

Recommendation JSON (Target crop to grow, detailed reasoning for selection).

### 3. ResourceAdvisor (**Fertilizer and Water Suggestions**)

This class relies on the output of the CropRecommender and queries a backend pricing database to plan resources.

#### **Attributes (Inputs):**

- + selectedCrop: String (*Sourced from CropRecommender*)
- + backendPriceList: PricingDatabase

#### **Methods (Operations):**

- + calculateResourceNeeds(): ResourcePlanReport
  - *Returns* ResourcePlanReport: An object detailing requiredFertilizers (List), totalCost (Float), waterQuantityRequired (Float/Volume), and recommendedIrrigationType (String).

### **Fertilizer and Water Need Suggestions**

#### **Actors Involved:**

- Farm Manager (Initiator)
- Resource Planning AI (Processor)
- Backend Pricing API (Data Source)

#### **Inputs:**

- Selected crop name (Output from Model 2)

#### **Functions (The Flow):**

1. The System receives the confirmed crop selection from the user.
2. The Resource Planning AI calculates the baseline lifecycle water and nutrient requirements for that specific crop.
3. The System determines the most efficient irrigation method based on the crop's water demands.
4. The System queries the Backend Pricing API to estimate the cost of the required fertilizers.
5. The System compiles the resource calculations and financial estimates into an advisory report.

**Outputs:**

Resource Plan Report (Required fertilizers, estimated costs, daily/weekly water quantity required, recommended irrigation type).

#### 4. GrowthTimelineGenerator

This class fuses historical timelines, visual data, and crop profiles to create a daily management schedule.

##### Attributes (Inputs):

- + dateSown: Date
- + cropData: CropProfile
- + qualityAssessmentImages: List<Image> (*Optional*)
- + diseaseDetectionImages: List<Image> (*Optional*)

##### Methods (Operations):

- + generateTimelineReport(): TimelineSchedule
  - *Returns TimelineSchedule:* A time-series report mapping out dailyWaterRequirement (Float), fertilizerType (String), fertilizerQuantity (Float), and applicationRate (Float) tailored to the current growth stage.

#### Growth Timeline for Water and Fertilizer Supply

##### Actors Involved:

- Farmer (Initiator/Executor)
- Timeline Scheduling AI (Processor)
- Crop Profile Database (Data Source)

##### Inputs:

- Automatic sowing date
- Standard crop growth data
- Recent quality assessment images (if available)
- Recent disease detection images (if available)

##### Functions (The Flow):

1. The System initializes a baseline timeline using the sowing date and standard crop profile.

2. The Timeline Scheduling AI ingests recent imagery to verify if the physical growth matches the expected timeline.
3. The System adjusts the schedule (accelerates or delays) based on observed visual data and past disease stress.
4. The System calculates the specific daily water volume and fertilizer application rate for the immediate upcoming growth phase.
5. The System publishes the updated operational schedule to the dashboard.

**Outputs:**

Timeline Schedule Report (Specific fertilizer types, quantities and application rates, adjusted daily water requirements).



## 5. QualityAssessor

This class evaluates the current state of the crop by comparing visual data against historical maintenance and disease records.

### Attributes (Inputs):

- + farmlandImages: List<Image>
- + diseaseHistory: List<DiseaseReport> (*Sourced from CropDiseaseDetector*)
- + maintenanceHistory: List<MaintenanceLog> (*Sourced from resource/water logs*)
- + daysInDevelopment: Integer

### Methods (Operations):

- + assessQuality(): QualityReport
  - *Returns* QualityReport: An object containing the currentQualityScore (Float or Grade) and improvementSuggestions (List of actionable steps based on how long the crop has been developing).

## Quality Assessment

### Actors Involved:

- Farm Worker (Initiator)
- Quality Assessment AI (Processor)
- Farm Logging System (Data Source)

### Inputs:

- Images from various zones of the farmland
- Disease history logs
- Maintenance history (water/fertilizer application logs)
- Days in development (Age of crop)

### Functions (The Flow):

1. The Farm Worker uploads current, multi-zone imagery of the field.
2. The Quality Assessment AI evaluates the visual health, color, and density against expected benchmarks for the crop's current age.

3. The System correlates the visual findings with past disease stresses and historical maintenance consistency.
4. The System calculates an overall quality score based on the combined data.
5. The System generates a list of actionable interventions to boost quality before harvest, provided there is still time in the development cycle.

**Outputs:**

Quality Report (Overall crop quality score, actionable suggestions for improvement).

## 6. YieldSegmenter (Crop Sell Quantity)

This class categorizes the final harvest into different market streams based on the cumulative data and final quality.

### Attributes (Inputs):

- + farmlandImages: List<Image>
- + diseaseHistory: List<DiseaseReport>
- + maintenanceHistory: List<MaintenanceLog>
- + cropQualityReport: QualityReport (*Sourced from QualityAssessor*)

### Methods (Operations):

- + calculateMarketDistribution(): SalesDistribution
  - *Returns* SalesDistribution: An object breaking down the yield into exportQuantity (Float), restaurantMarketplaceQuantity (Float), and foodProcessingQuantity (Float).

## Crop Sell Quantity (Yield Segmentation)

### Actors Involved:

- Farmer / Sales Manager (Initiator)
- Yield Segmentation AI (Processor)

### Inputs:

- Final pre-harvest farmland images
- Complete disease history
- Complete maintenance history
- Final Quality Assessment Report

### Functions (The Flow):

1. The System aggregates all historical logs, final imagery, and the terminal quality score.
2. The Yield Segmentation AI predicts the total harvest volume based on crop density in the final images.

3. The System cross-references the predicted volume and quality score against strict market tier requirements.
4. The System partitions the total predicted yield, assigning premium batches to export, standard to local markets, and flawed/lower-tier batches to processing.
5. The System finalizes the volume distribution for logistical planning.

**Outputs:**

- Sales Distribution Report (Quantity optimized for export, quantity for restaurants/marketplaces, quantity for food processing).

## 7. LogisticsTransportationOptimizer

This class ingests the target sales distribution quantities and coordinates the most cost-effective, time-sensitive delivery matrix across an available fleet.

### Attributes (Inputs):

- + marketDistribution: SalesDistribution (*Sourced from Model 6: Crop Sell Qty*)
- + destinationHubs: List<Location> (*Coordinates for Export Ports, Restaurants, and Processing Plants*)
- + availableFleet: List<Vehicle> (*Capacities, fuel consumption rates, and cold-chain/refrigeration capabilities*)
- + realTimeTrafficAndWeather: LiveLogisticsData
- + cropPerishabilityIndex: Float (*Decay rate of the crop under various temperatures*)

### Methods (Operations):

- + optimizeRoutesAndFleet(): LogisticsPlan
  - *Returns LogisticsPlan*: An object detailing fleetAssignments (which truck goes where), optimizedWaypoints (GPS routing structures), estimatedDeliveryTimes (ETAs), and minimizedLogisticsCost (Float).

## Flow Diagram & Operational Mechanics

### Actors Involved:

- Logistics Manager (Initiator / Fleet Dispatcher)
- Fleet Routing AI Engine (Processor)
- External Telematics API (Live traffic, road closures, and weather)

### Inputs:

- Quantities assigned to Export, Marketplaces, and Processing (Output of Model 6)
- Fleet status (refrigerated vs. non-refrigerated trucks, capacities)
- Destination distances and current road constraints

### Functions (The Flow):

1. The Logistics Manager initializes the dispatch system after a harvest allocation is finalized.

2. The System queries the Fleet Status database for active drivers, truck capacities, and fuel metrics.
3. The System fetches live traffic congestion, construction alerts, and weather patterns along potential transit paths.
4. The Fleet Routing AI checks the cropPerishabilityIndex. Premium export crops requiring cold-chain transportation are automatically prioritized for refrigerated vehicles.
5. The AI runs optimization algorithms (like Genetic Algorithms or Deep Reinforcement Learning) to minimize overall fuel burn and transit time while eliminating the risk of crop spoilage.
6. The System outputs a dynamic dispatch schedule.

**Outputs:**

- Logistics Plan (Assigned trucks, exact GPS waypoints, prioritized departure times, and total cost breakdown).